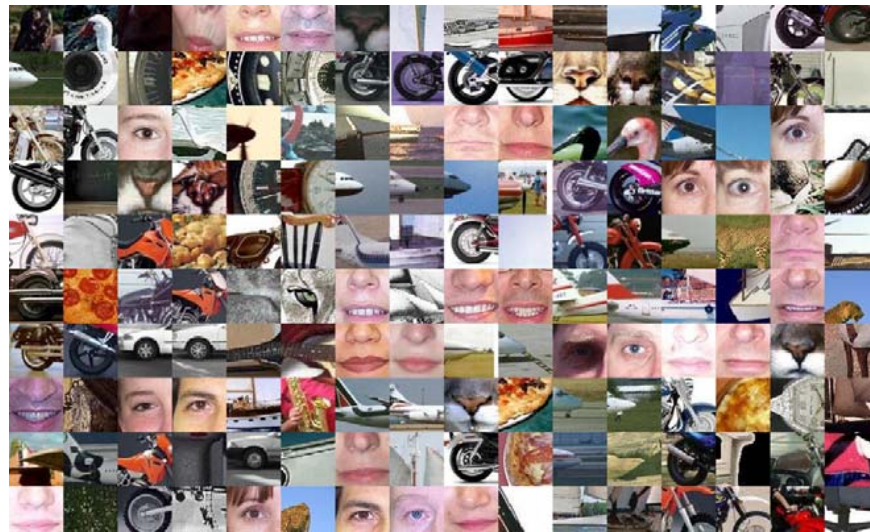
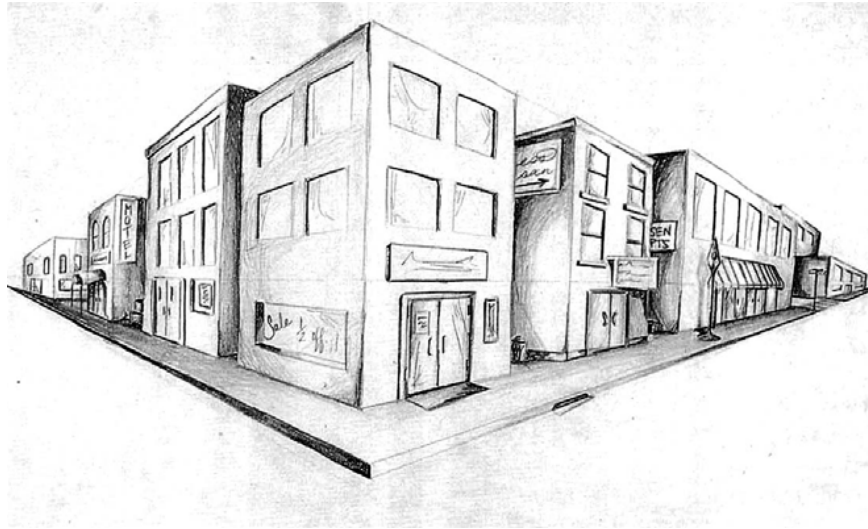


Feature detectors and descriptors



Slide Credits: Ioannis Gkioulekas, Kris Kitani, Fredo Durand, James Hays

Corner detector





Here are the corners detected overlaid on the image. You can see that many corners were detected at the same place for both viewpoints. This is good because it will allow us to match between the images. Still not all the detected features match, so the next part of the algorithm will have to be robust to this. We will see later how.

Laplacian feature detection

So far we have seen the Harris corner detector. Now let's discuss a simpler type of feature.

Laplacian.

(but the same idea applies to Harris).

For various historical reasons, the popular SIFT descriptor we see next uses Laplacian keypoints rather than Harris.

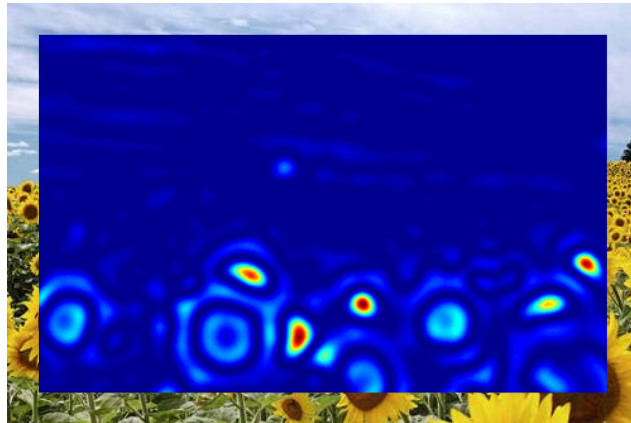
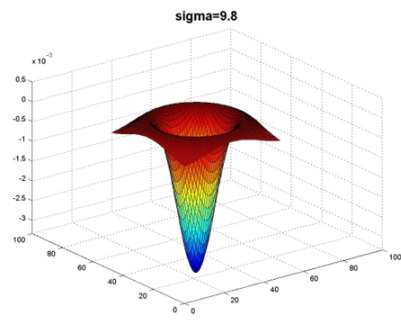
What does this filter like?

In computer vision there are often multiple ways to achieve similar results, and there is not always one good solution.

We discussed the Harris corner detector. But we will see now a different detector which made it way into the popular SIFT descriptor.

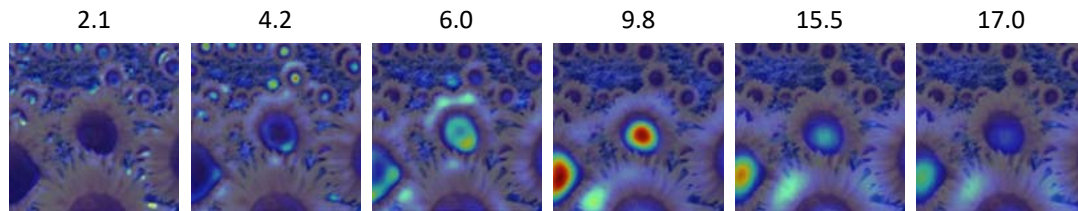
We will use it to demonstrate the idea of multi scale detection.

We can also do multi scale detection of the Harris operator, but the Laplacian detector is a bit easier to visualize.

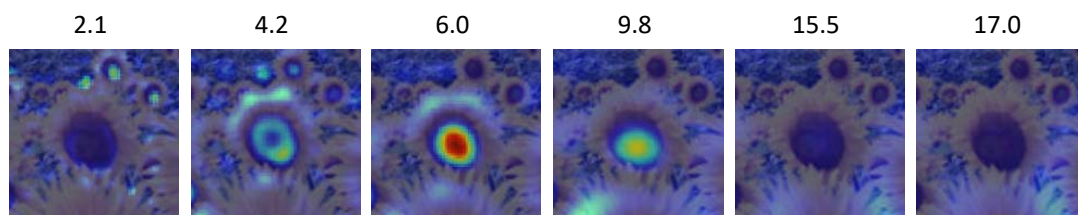


Multi-scale detection

optimal scale



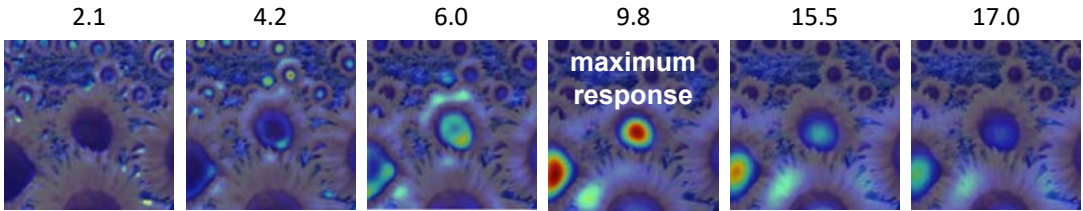
Full size image



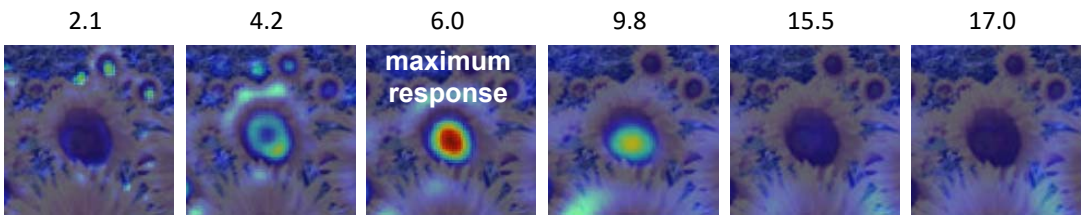
3/4 size image

If we scale the image, a Laplacian filter at a different width will fire, namely a different scale of the Laplacian detector will detect that feature.

optimal scale

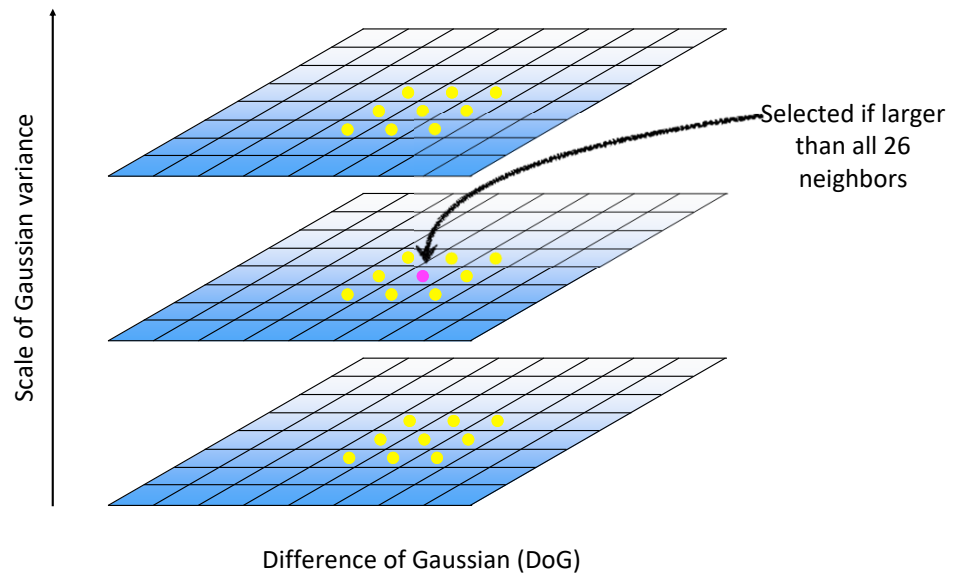


Full size image

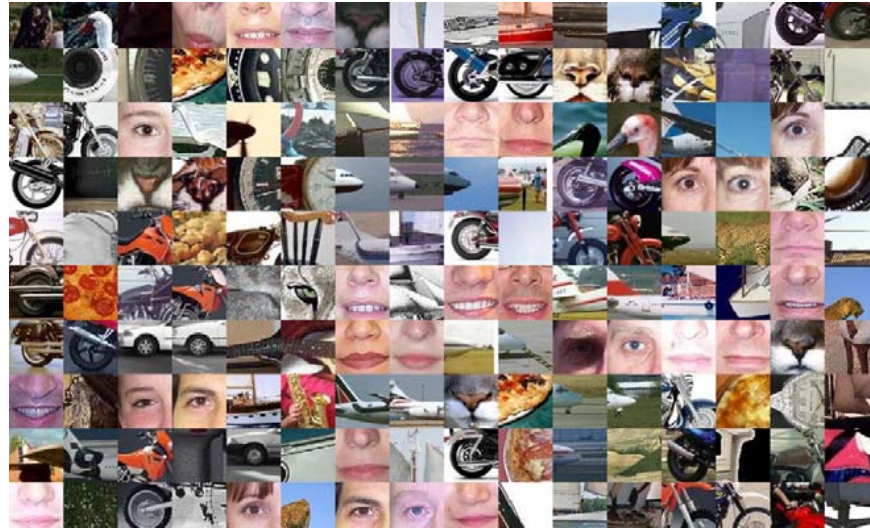


3/4 size image

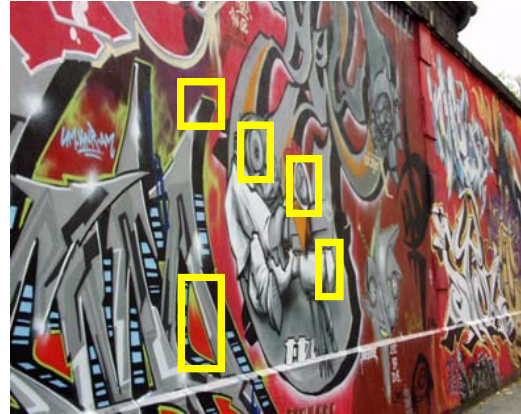
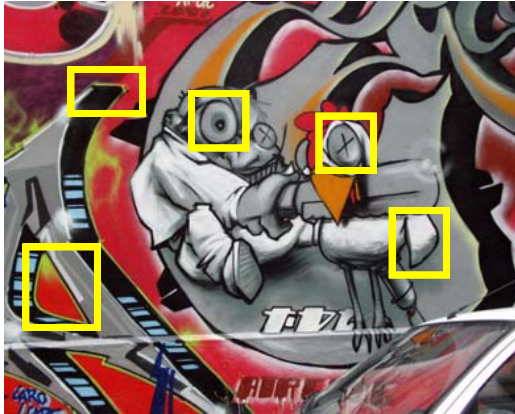
Scale-space extrema



Feature descriptors



Why do we need feature
descriptors?



*If we know where the good features are,
how do we match them?*

So far we have seen how to detect features.

To match between images, we also need to match between the features detected at different images.

The right way to match them is not to compare their intensities (the intensities can be very different), but look for some description (some summarizing representation) which is invariant to various deformations (rotation, intensity, scale...).

Note so far, we discussed how to make the detector invariant to deformation. But after we found it, we also need to describe it in a deformation invariant way.

Object instance recognition



Schmid and Mohr 1997



Sivic and Zisserman, 2003



Rothganger et al. 2003

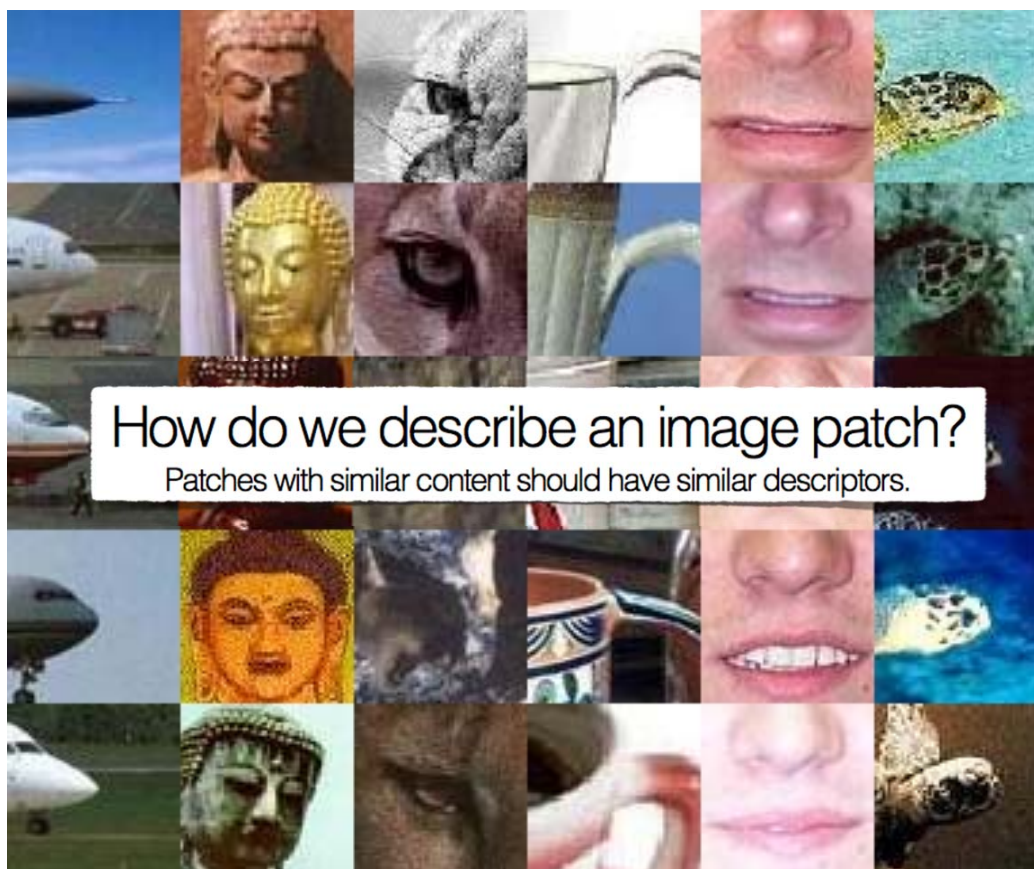


Lowe 2002

Image mosaicing



הערות: מציבים את העדשה במצב
auto-exposure (מצב עצום)
על מנת למנוע שינויים בצבעי האדום



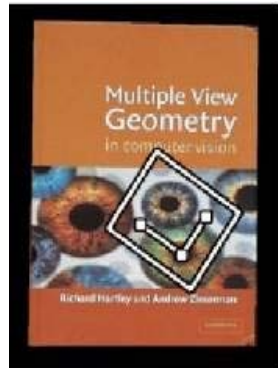
Designing feature descriptors

So we want to design a good descriptor. Let's start by thinking what invariance we want.

Photometric transformations



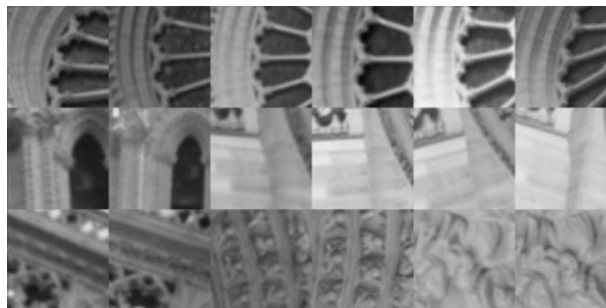
Geometric transformations



objects will appear at different scales,
translation and rotation



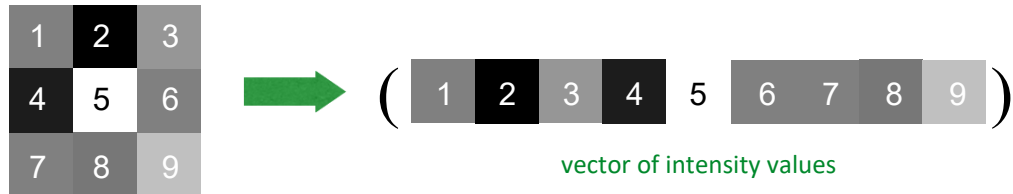
What is the best descriptor for an image feature?



Let's see some possible approaches for describing an image patch, and let us think if they are good or bad.

Image patch

Just use the pixel values of the patch



Perfectly fine if geometry and appearance is unchanged (a.k.a.
template matching)

Tiny Images



Just down-sample it!
Simple, fast, robust to small affine transforms.



Image patch

Just use the pixel values of the patch



Perfectly fine if geometry and appearance is unchanged (a.k.a. template matching)

What are the problems?



Image patch

Just use the pixel values of the patch



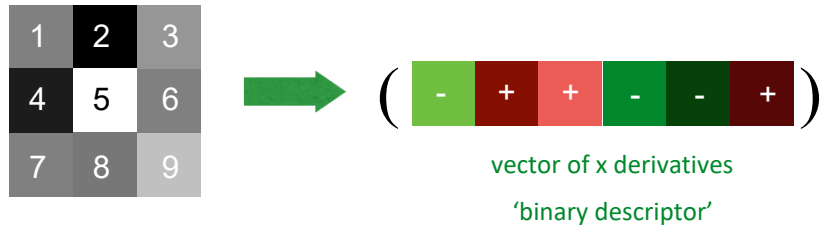
Perfectly fine if geometry and appearance is unchanged (a.k.a.
template matching)

What are the problems?

How can you be less sensitive to absolute intensity values?

Image gradients

Use pixel differences



Feature is invariant to absolute intensity values

What are the problems?

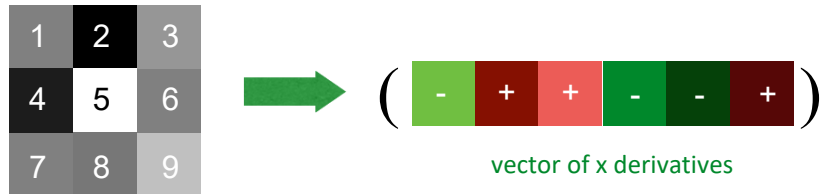
Remember we described local intensity changes as $I(xy) \rightarrow aI(xy) + b$

Derivatives are invariant to adding b.

If we only keep the sign of the derivatives (negative or positive), we are also invariant to scaling by a.

Image gradients

Use pixel differences



Feature is invariant to absolute intensity values

What are the problems?

How can you be less sensitive to deformations?

But this is still not invariant to deformations, such as a rotation of the patch.

Color histogram

Count the colors in the image using a histogram



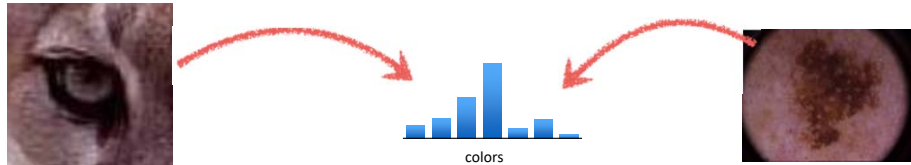
Invariant to changes in scale and rotation

What are the problems?

One way to make our descriptor invariant to rotation is to store only its color histogram

Color histogram

Count the colors in the image using a histogram



Invariant to changes in scale and rotation

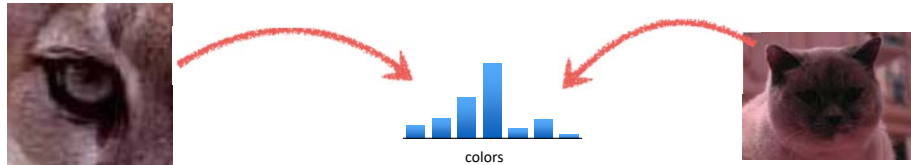
What are the problems?

Descriptor should not only be deformation invariant but also discriminative

But then we can find very different images, which still have the same histogram.
We do not want to say that the two images on this slide are the same.

Color histogram

Count the colors in the image using a histogram



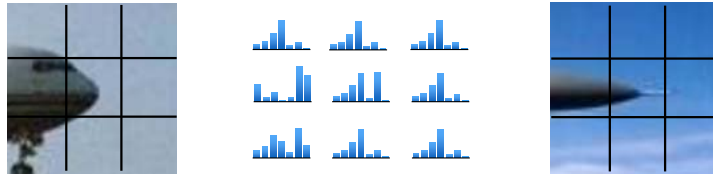
Invariant to changes in scale and rotation

What are the problems?

How can you be more sensitive to spatial layout?

Spatial histograms

Compute histograms over spatial 'cells'

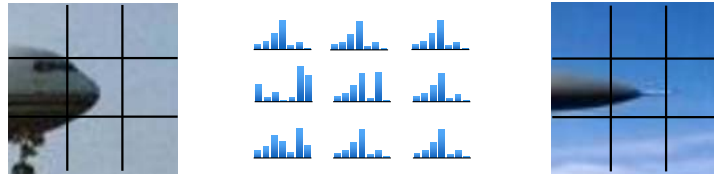


Retains rough spatial layout
Some invariance to deformations

What are the problems?

Spatial histograms

Compute histograms over spatial 'cells'



Retains rough spatial layout
Some invariance to deformations

What are the problems?

How can you be completely invariant to rotation?

Orientation normalization

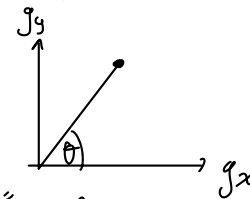
Use the dominant image gradient direction to normalize the orientation of the patch



save the orientation angle θ along with (x, y, s)
for each pixel:

$$g_x = [-1, 1] * I$$

$$g_y = [-1, 1] * I$$



"تجهيز" "تجهيز" "تجهيز"

To make our descriptor invariant to rotation we detect the direction with strongest gradient and rotate the patch such that this direction will be e.g. horizontal.

This is a very important idea that we will see a lot in the coming slides.

Remember: to make the descriptor rotation invariant we find the dominant orientation and rotate the patch accordingly.

We will rotate all patches BEFORE we match them.

SIFT descriptor

SIFT is a way to describe an image window containing a feature of interest, in a way that it is invariant to small changes/deformations in the content, orientation, illumination. This is one of the more successful ideas that have influenced computer vision research quite a lot. After that we will see a few later versions of descriptors, all inspired by similar ideas.



SIFT

(Scale Invariant Feature Transform)

SIFT describes both a **detector** and **descriptor**

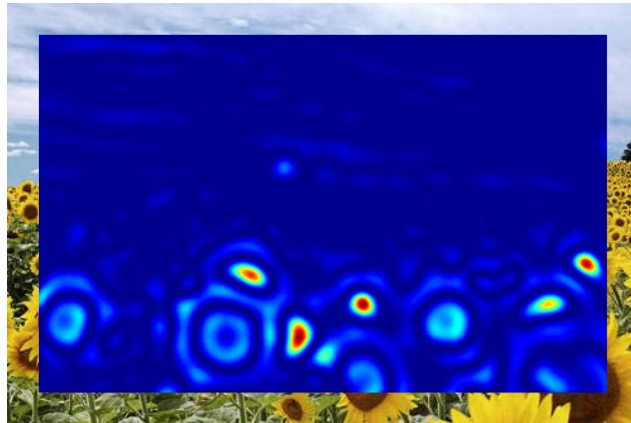
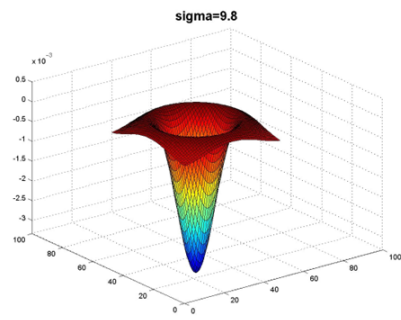
Laplacian multi-scale

1. Multi-scale extrema detection
2. Keypoint localization
3. Orientation assignment
4. Keypoint descriptor

Discussed in
tutorial

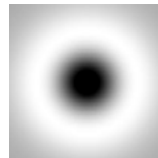
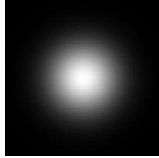
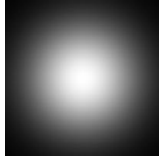
SIFT starts by detecting keypoints (features) of interest. We have covered these stages earlier, but will review in the coming slides.

Once it detected a feature it builds a summarized representation which we will explain next.



Difference of Gaussians

$$G_{\sigma_1} * I - G_{\sigma_2} * I = (G_{\sigma_1} - G_{\sigma_2}) * I$$



Laplacian

Q:What does this filter like?

Bloby circular image areas



Gaussian



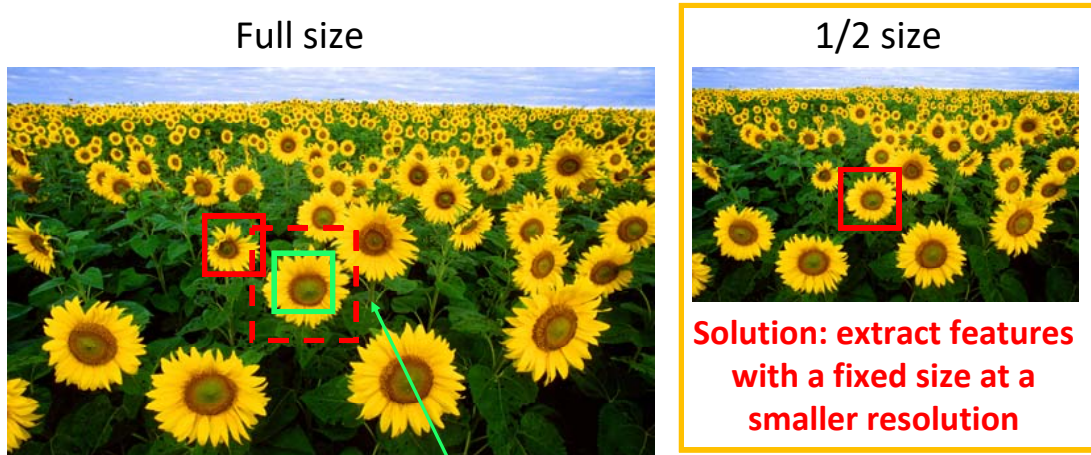
Laplacian

The first stage is to detect keypoints. We use a Laplacian (difference of Gaussian) filter, convolve it with the image and detect local extrema.

To apply this we convolve the image with multiple scales of the Laplacian filter and store in a pyramid.

When the scale gets wide we also subsample the image. This sampling has 2 motivations: 1) computational efficiency. 2) We will effectively include in our window more content.

Use multiple octaves to extract the feature at the proper scale and proper window support



Features detected at 2 different scales and their desired support.

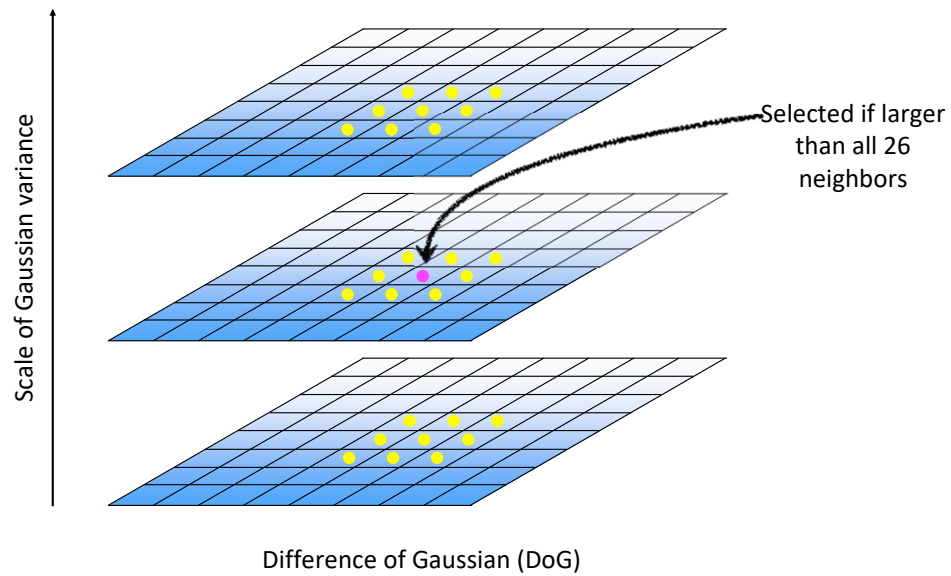
If we extract windows with a fixed size, window will be too small.

If we detect the feature at a higher scale we need to extract a wider window around it to include the same amount of information.

However, for the next stages it is simpler if all the windows we extract include the same number of pixels.

The solution is to downsample the image and extract a window with a fixed number of pixels, but from a lower resolution.

Scale-space extrema



We detect a Laplacian feature if its response is bigger/smaller than all of its neighbors. As a neighbor we consider a 2D shift, and also neighbors in scale.

3. Orientation assignment

For a keypoint, **L** is the **Gaussian-smoothed** image with the closest scale,

$$m(x, y) = \sqrt{(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2}$$

x-derivative y-derivative

$$\theta(x, y) = \tan^{-1}((L(x, y+1) - L(x, y-1)) / (L(x+1, y) - L(x-1, y)))$$

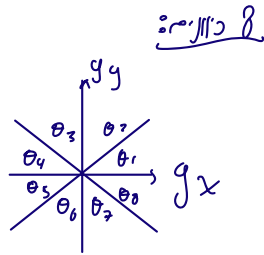
Detection process returns

$$\{x, y, \sigma, \theta\}$$

location scale orientation

Once we detect a feature we find the dominant orientation and rotate it accordingly.

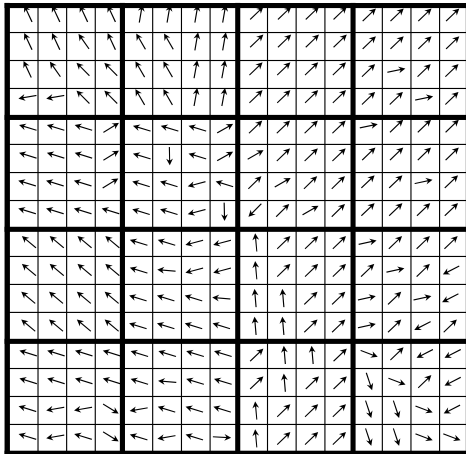
For each feature we detect we store 4 parameters. First the x,y position where it was detected, then the scale, and finally the orientation.



4. Keypoint descriptor

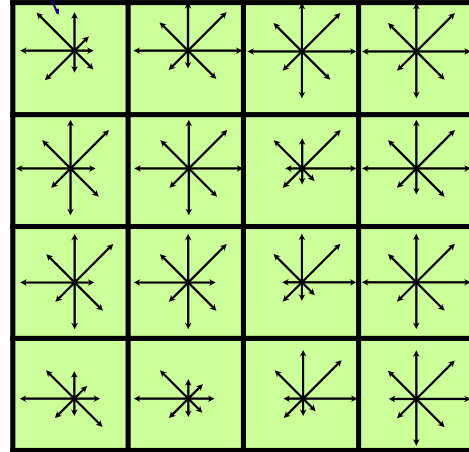
Image Gradients

(4 x 4 pixel per cell, 4 x 4 cells)



SIFT descriptor

(16 cells x 8 directions = 128 dims)



So far we discuss the stages involved in the detection of the feature. Once we detected it, we need to describe its content in a way that will be invariant to various deformations.

The descriptor works as follows: First we compute the gradient orientation at each pixel. Assume the window we extract contains 16x16 pixels. We divide them into 4x4 subwindows, each window contains 4x4 pixels. In each window we form a histogram of orientations. We divide the orientations into 8 discrete directions. The direction of every gradient we compute in the image is rounded to one of these 8 orientations.

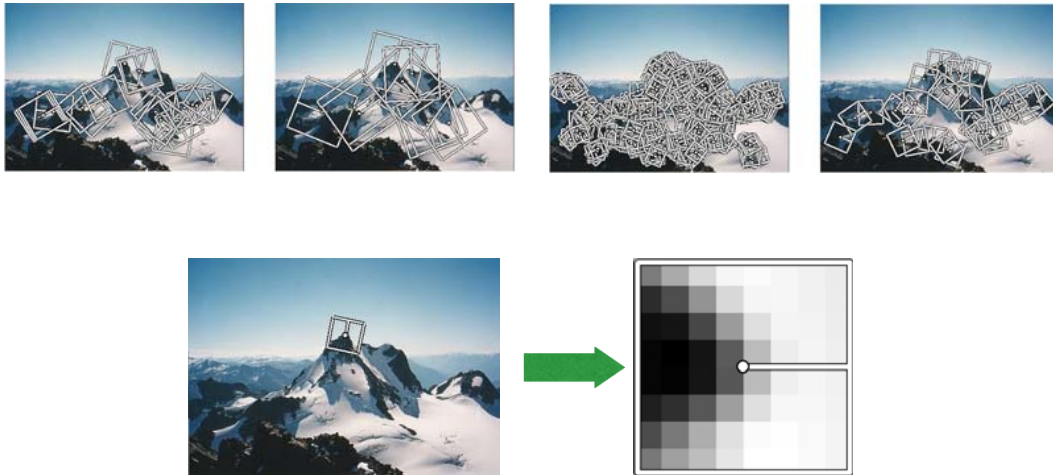
Then we store a vector of 8 numbers, in each bin we keep the number of pixels (out of our 4x4 block) with this orientation.

Overall we have $4 \times 4 = 16$ subwindows and in each we keep a histogram of 8 bins. So in total we describe the patch by $16 \times 8 = 128$ numbers.

Important: multi-scale oriented application

Note that feature histograms are computed at the scale at which the feature was detected. (coarser scale- histograms on a larger window, or use image at a higher octave)

Edge orientation histograms: again with respect to canonical orientation



Invariances

SIFT descriptor is invariant to:

אנחנו רוצים שההופכי של $I(x,y)$ יהיה זהה ל- $I(x,y)$
כלומר, אנחנו רוצים שההופכי של $I(x,y)$ יהיה זהה ל- $I(x,y)$
זה נובע מכך שאנחנו משתמשים בגרדיינטים

Illumination: because we only use gradients →

Rotation and scale: We align orientations and extract window from the corresponding octave.

Small variation in appearance/shape: we summarize each 4x4 window by a histogram, invariant to small changes inside this window.

או down-sample into 2x2

GIST descriptor

In the coming slides we will see a few other descriptors. They are all based on similar ideas.

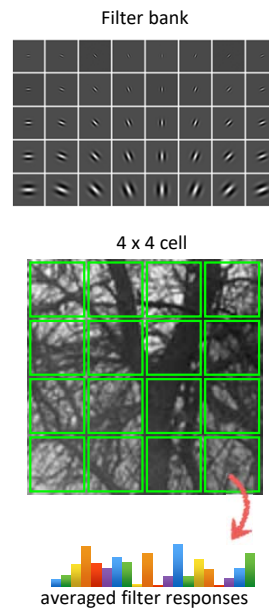
We review them to give you the sense that in computer vision there are often multiple ways to achieve similar results, and there is no unique best answer.

In many algorithms small engineering details such as the exact thresholds you select may make a big difference.

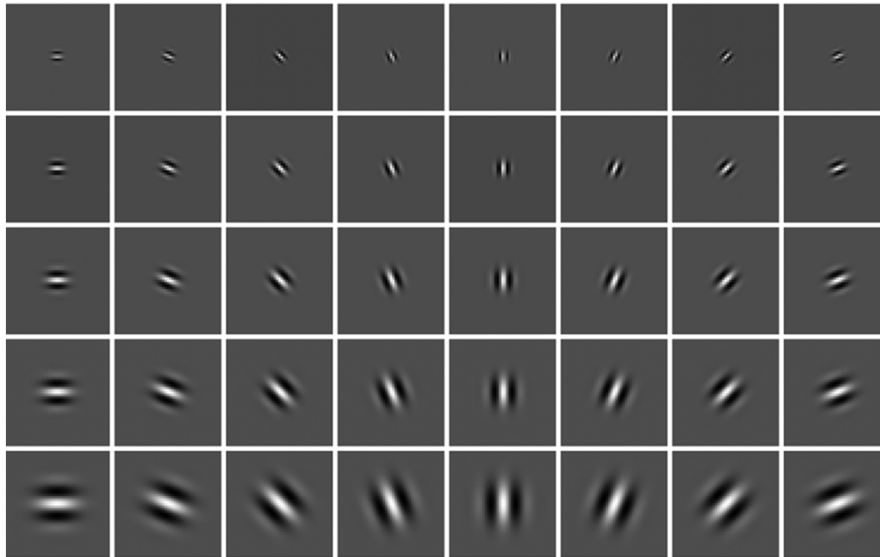
GIST is another type of descriptor invented after the original SIFT descriptor, uses similar ideas.

GIST

1. Compute filter responses (filter bank of Gabor filters)
2. Divide image patch into 4 x 4 cells
3. Compute filter response averages for each cell
4. Size of descriptor is $4 \times 4 \times N$, where N is the size of the filter bank



Directional edge detectors

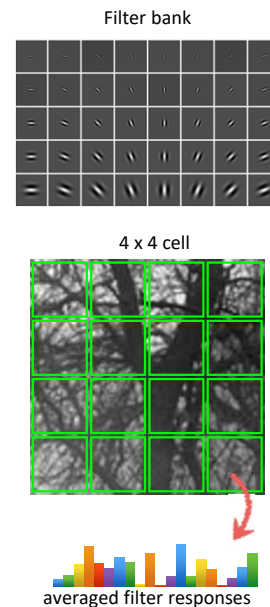


GIST

1. Compute filter responses (filter bank of Gabor filters)
2. Divide image patch into 4 x 4 cells
3. Compute filter response averages for each cell
4. Size of descriptor is 4 x 4 x N, where N is the size of the filter bank

What is the GIST descriptor encoding?

Rough spatial distribution of image gradients



Difference between GIST and SIFT:

1) In SIFT we find one of 8 orientations closest to the orientation of each gradient and count it only in this bin.

In GIST we convolve the image with filters in different orientations and sum the response of each filter. Effectively a pixel may contribute to multiple orientations.

2) Filters at multiple scales, adds robustness. In SIFT only local gradients are considered.

MOPS descriptor

MOPS is yet another descriptor with similar principles.

Multi-Scale Oriented Patches (MOPS)

Multi-Image Matching using Multi-Scale Oriented Patches. M. Brown, R. Szeliski and S. Winder.
International Conference on Computer Vision and Pattern Recognition (CVPR2005). pages 510-517



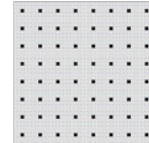
Multi-Scale Oriented Patches (MOPS)

Multi-Image Matching using Multi-Scale Oriented Patches. M. Brown, R. Szeliski and S. Winder.
International Conference on Computer Vision and Pattern Recognition (CVPR2005). pages 510-517

Given a feature (x, y, s, θ)

Get 40 x 40 image patch, subsample
every 5th pixel

(what's the purpose of this step?)



Subtract the mean, divide by standard
deviation

(what's the purpose of this step?)

Haar Wavelet Transform

(what's the purpose of this step?)

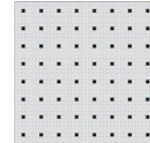
Multi-Scale Oriented Patches (MOPS)

Multi-Image Matching using Multi-Scale Oriented Patches. M. Brown, R. Szeliski and S. Winder.
International Conference on Computer Vision and Pattern Recognition (CVPR2005). pages 510-517

Given a feature (x, y, s, θ)

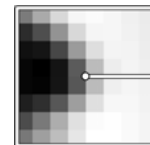
Get 40 x 40 image patch, subsample
every 5th pixel

(low frequency filtering, absorbs localization errors)



Subtract the mean, divide by standard
deviation

(what's the purpose of this step?)



Haar Wavelet Transform

(what's the purpose of this step?)

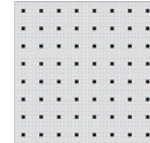
Multi-Scale Oriented Patches (MOPS)

Multi-Image Matching using Multi-Scale Oriented Patches. M. Brown, R. Szeliski and S. Winder.
International Conference on Computer Vision and Pattern Recognition (CVPR2005). pages 510-517

Given a feature (x, y, s, θ)

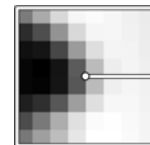
Get 40 x 40 image patch, subsample
every 5th pixel

(low frequency filtering, absorbs localization errors)



Subtract the mean, divide by standard
deviation

(removes bias and gain)



Haar Wavelet Transform

(what's the purpose of this step?)



So far the ideas are similar to previous descriptors- we bring all features to a canonical orientation and we normalize intensity to be invariant to illumination.

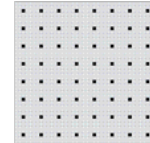
Multi-Scale Oriented Patches (MOPS)

Multi-Image Matching using Multi-Scale Oriented Patches. M. Brown, R. Szeliski and S. Winder.
International Conference on Computer Vision and Pattern Recognition (CVPR2005). pages 510-517

Given a feature (x, y, s, θ)

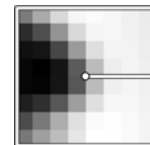
Get 40 x 40 image patch, subsample
every 5th pixel

(low frequency filtering, absorbs localization errors)



Subtract the mean, divide by standard
deviation

(removes bias and gain)



Haar Wavelet Transform

(low frequency projection)

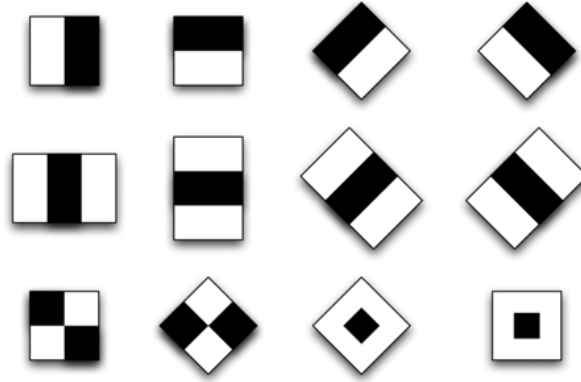


The Haar transform is yet another way to compute a histogram of orientations

Haar Wavelets

(actually, Haar-like features)

Use responses of a bank of filters as a descriptor



Can compute Haar wavelet responses **efficiently** (in constant time) with integral images

The advantage of Haar wavelets over the filter bank of the previous descriptor is that it can be computed more efficiently.

BRIEF descriptor

Binary Robust Independent
Elementary Features

A simple descriptor, to be implemented in assignment 1.

BRIEF Binary Robust Independent Elementary Features

Michael Calonder, Vincent Lepetit, Christoph Strecha, and Pascal Fua. ECCV2010

Select a set of $n=256$ pairs (x,y) in patch I .

Store a binary descriptor

$$\rho_n(I, x_n, y_n) = \begin{cases} 1 & \text{if } I(x_n) < I(y_n) \\ 0 & \text{otherwise} \end{cases}$$

Invariant to affine intensity transformation?

$$\rho_n(I, x_n, y_n) = \rho_n(aI + b, x_n, y_n)$$

Invariant to scale?

Apply at characteristic scale of the interest point

Invariant to rotation?

Nope

Robustness to noise?

Smooth image before testing the “<” relations

256 pairs and for each a binary value- effectively we encode each patch by 1 character.

The BRIF descriptor is very simple and efficient to implement. Descriptors we described earlier have better performances.

BRIEF Binary Robust Independent Elementary Features

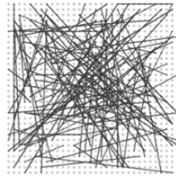
Michael Calonder, Vincent Lepetit, Christoph Strecha, and Pascal Fua. ECCV2010

Select a set of $n=256$ pairs (x,y) in patch I .

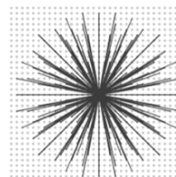
Store a binary descriptor

$$\rho_n(I, x_n, y_n) = \begin{cases} 1 & \text{if } I(x_n) < I(y_n) \\ 0 & \text{otherwise} \end{cases}$$

Which pairs to select?



Random



Polar grid

References

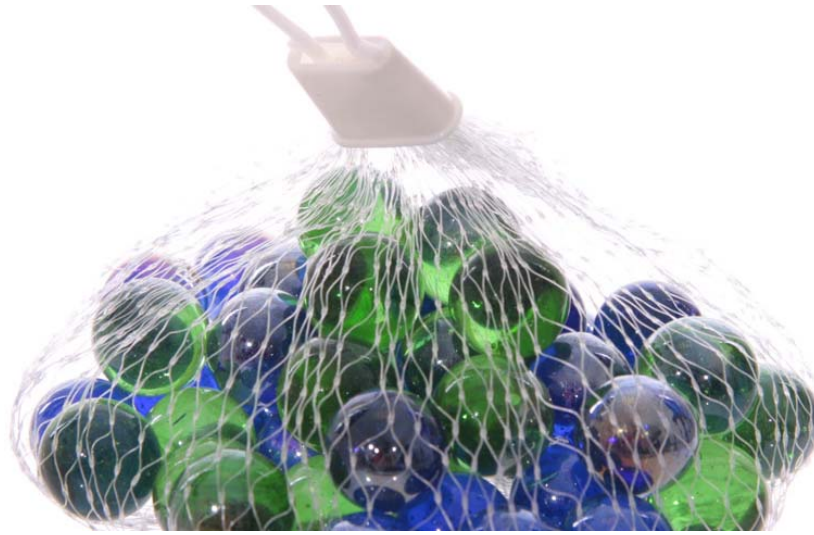
Basic reading:

- Szeliski textbook, Sections 4.1.

Image descriptors

- SIFT descriptors describe patches. How do we describe a big image?

Image classification



- Slide credits Ioannis Gkioulekas Kris Kitani, Noah Snavely, Fei-Fei Li

Image Classification

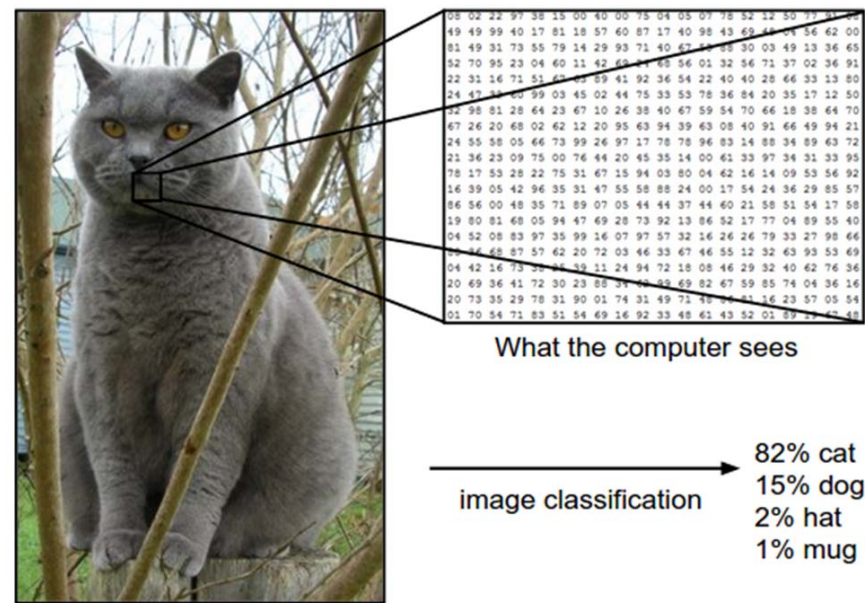


(assume given set of discrete labels)
{dog, cat, truck, plane, ...}



cat

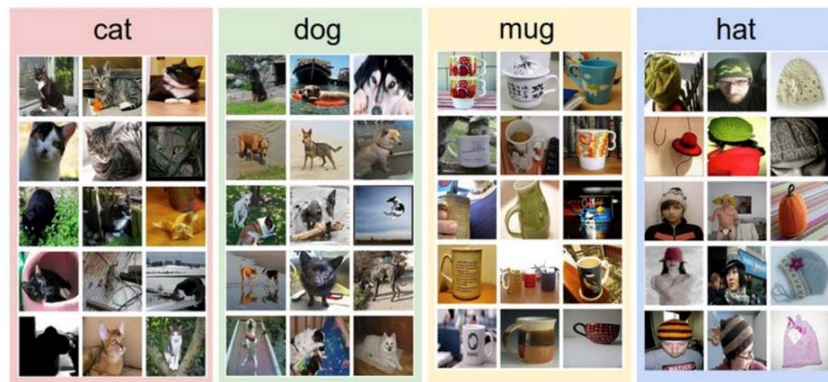
Image Classification: Problem



Data-driven approach

- Collect a database of images with labels
- Use ML to train an image classifier
- Evaluate the classifier on test images

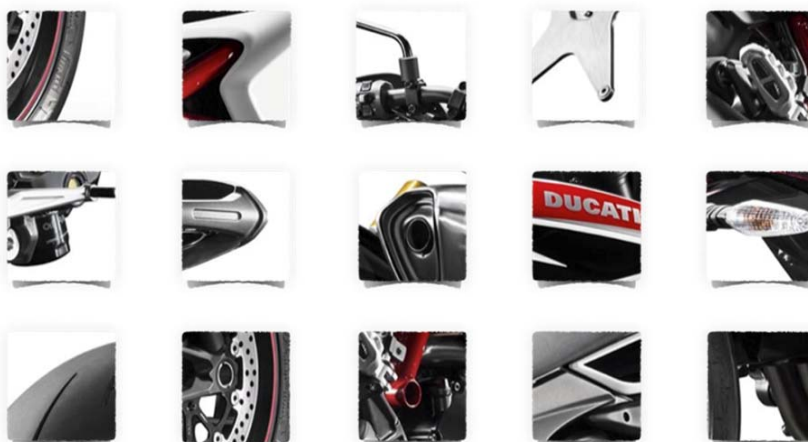
Example training set



In computer vision, the name of the game is data, data, and more data.

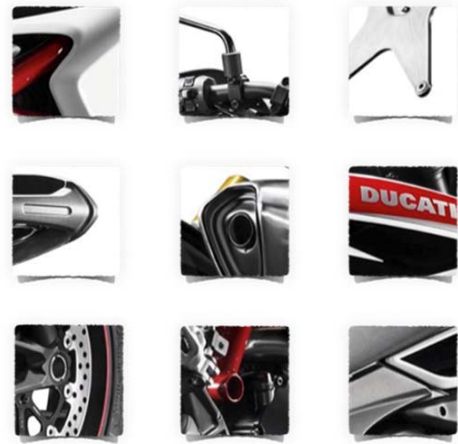
Bag of words

What object do these parts belong to?



Some local feature are
very informative

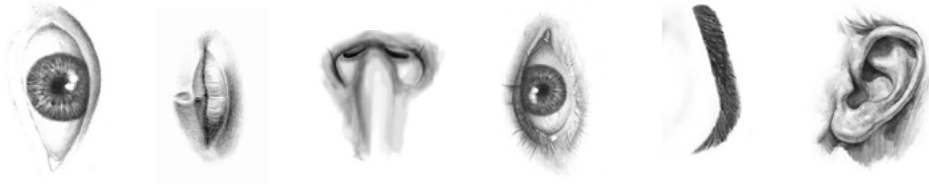
An object as



a collection of local features
(bag-of-features)

- deals well with occlusion
- scale invariant
- rotation invariant
- Deformations invariant

(not so) crazy assumption



spatial information of local features
can be ignored for object recognition (i.e., verification)

CalTech6 dataset



class	bag of features	bag of features	Parts-and-shape model
	Zhang et al. (2005)	Willamowski et al. (2004)	Fergus et al. (2003)
airplanes	98.8	97.1	90.2
cars (rear)	98.3	98.6	90.3
cars (side)	95.0	87.3	88.5
faces	100	99.3	96.4
motorbikes	98.5	98.0	92.5
spotted cats	97.0	—	90.0

Works pretty well for image-level classification

Csurka et al. (2004), Willamowski et al. (2005), Grauman & Darrell (2005), Sivic et al. (2003, 2005)

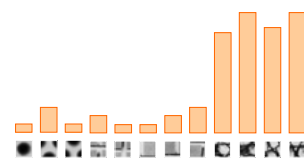
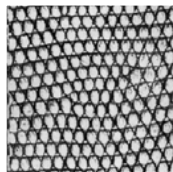
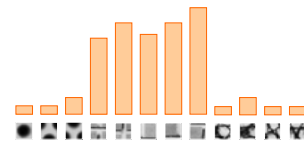
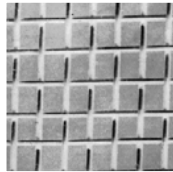
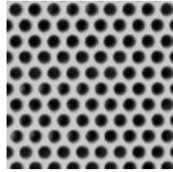
Bag-of-features

represent a data item (document, texture, image)
as a histogram over features

an old idea

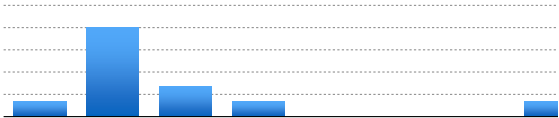
e.g., texture recognition and information retrieval (text documents)

Texture recognition

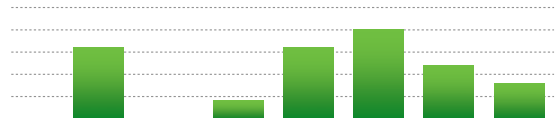


Vector Space Model

G. Salton. 'Mathematics and Information Retrieval' Journal of Documentation,1979



1	6	2	1	0	0	0	1
Tartan	robot	CHIMP	CMU	bio	soft	ankle	sensor



0	4	0	1	4	5	3	2
Tartan	robot	CHIMP	CMU	bio	soft	ankle	sensor

Standard BOW pipeline

(for image classification)

Dictionary Learning:

Learn Visual Words using clustering

Encode:

build Bags-of-Words (BOW) vectors
for each image

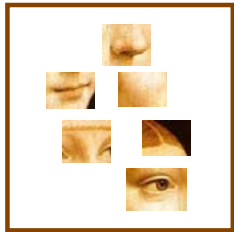
Classify:

Train and test data using BOWs

Dictionary Learning:

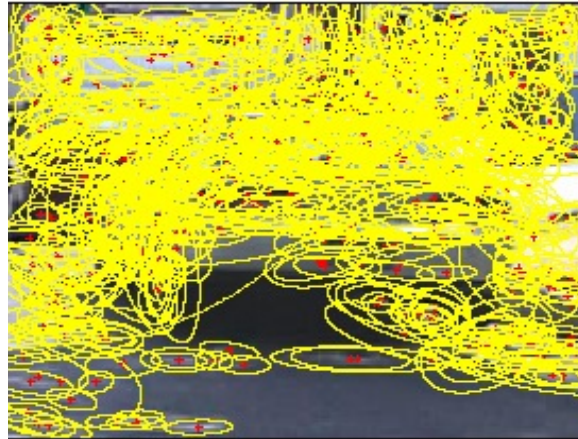
Learn Visual Words using clustering

1. extract features (e.g., SIFT) from images



What kinds of features can we extract?

- **Regular grid**
 - Vogel & Schiele, 2003
 - Fei-Fei & Perona, 2005
- **Interest point detector**
 - Csurka et al. 2004
 - Fei-Fei & Perona, 2005
 - Sivic et al. 2005
- **Other methods**
 - Random sampling (Vidal-Naquet & Ullman, 2002)
 - Segmentation-based patches (Barnard et al. 2003)



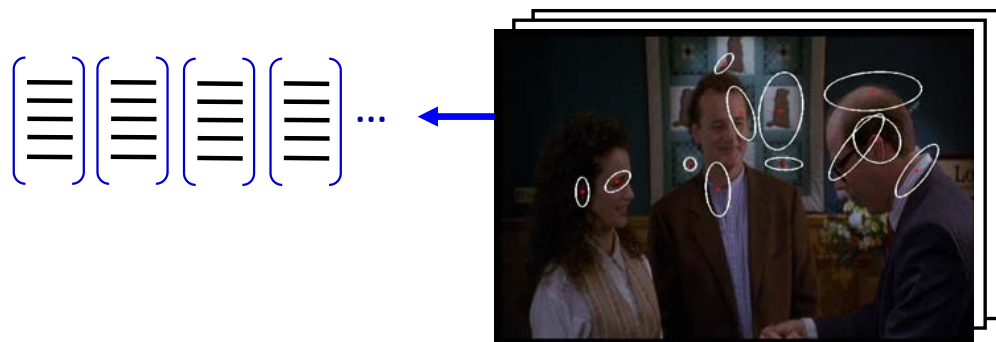
There are many ways to select features in an image. You can select interest points using e.g. a Harris detector.

In practice people have suggested also more naïve extraction of features, e.g. just crop random patches from images, or using a grid of all patches.

describing the features- SIFT



Once we have features we will want to compare features in different images and identify they are similar. Our features are just image patches. If we express them as arrays of numbers, things we identify as similar may be very different from the computer viewpoint. As discussed in the previous part, describing a patch using a SIFT descriptor is better, because this representation is invariant to some deformations (illumination, rotation, scale, some small perturbations).



From each image we extract multiple features and we express each of them as a vector of 128 numbers corresponding to entries of the SIFT descriptor

Dictionary Learning: Learn Visual Words using clustering

2. Learn visual dictionary (e.g., K-means clustering)

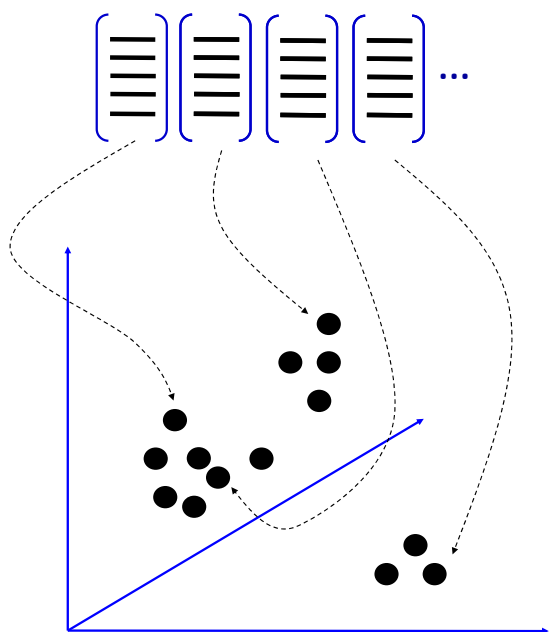


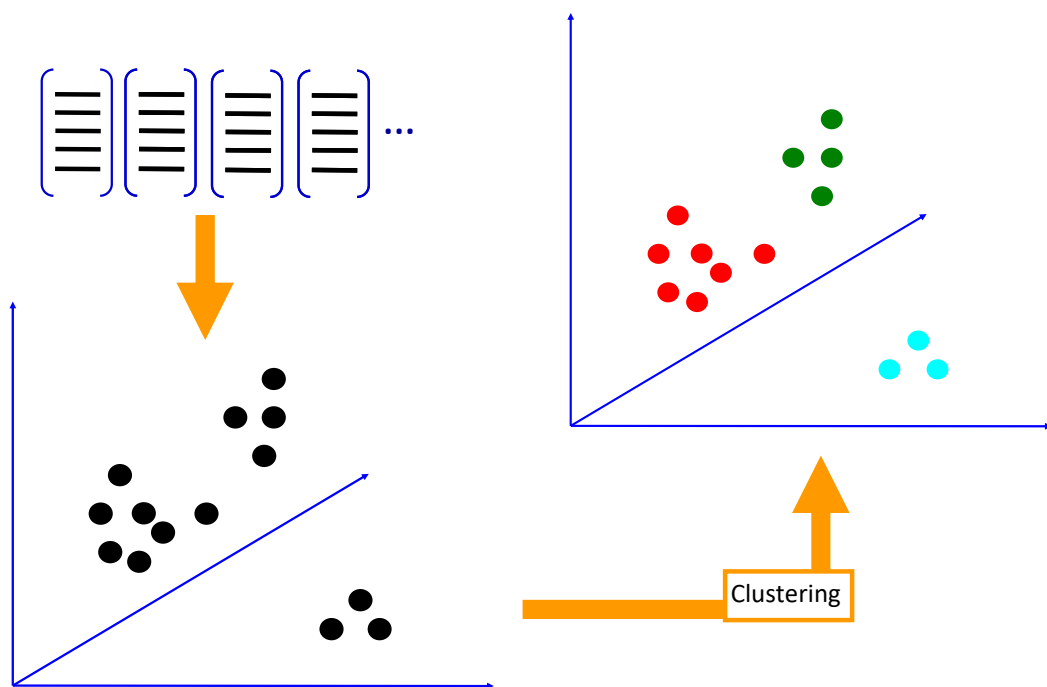
As part of our classification algorithms we will want to compare the words extracted from different images.

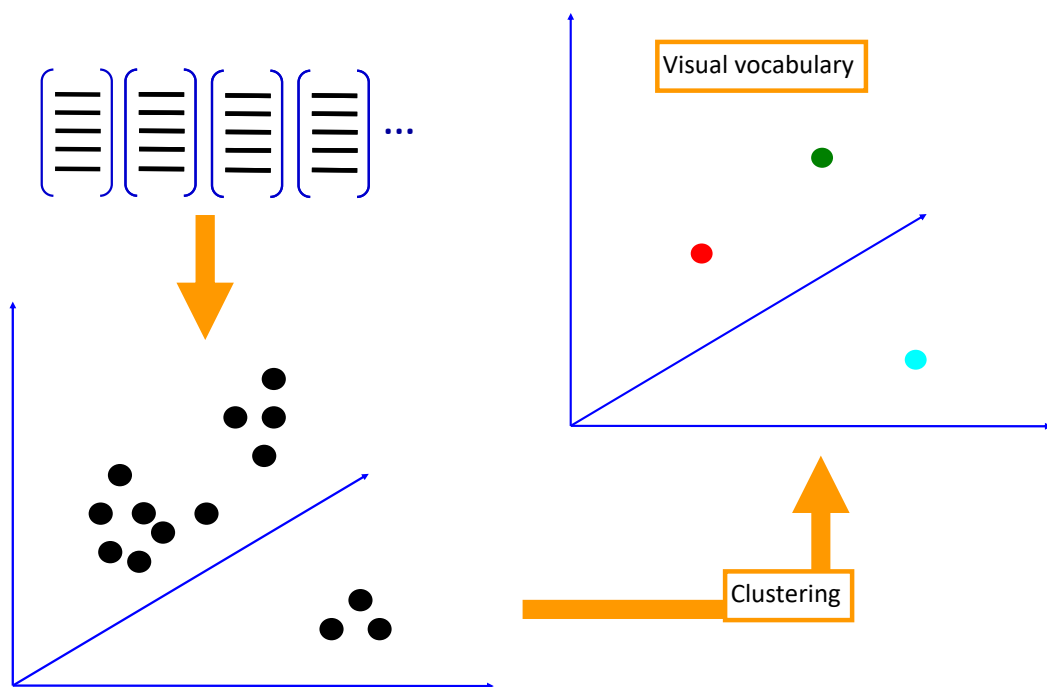
Unlike text, we will never get exactly the same word twice in two different images. Even if we use a SIFT descriptor the probability that two patches will provide exactly the same descriptor is very low. We want to be able to say that if two features are similar, they count as the same word.

For that we divide all the features we extracted to clusters. Two different features will count as the same “word” if they belong to the same cluster

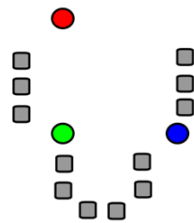
How do we learn the dictionary?



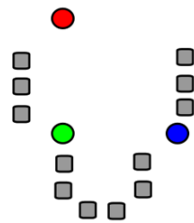




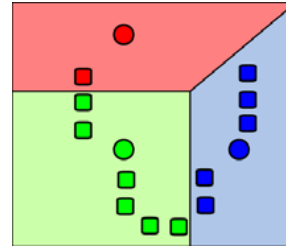
K-means clustering



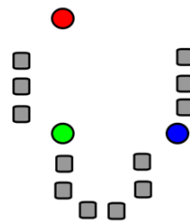
1. Select initial
centroids at random



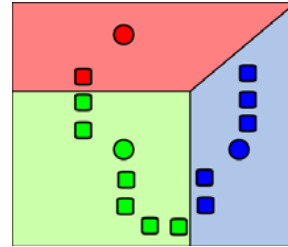
1. Select initial centroids at random



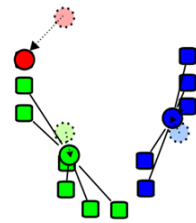
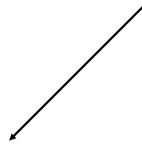
2. Assign each object to the cluster with the nearest centroid.



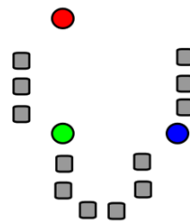
1. Select initial centroids at random



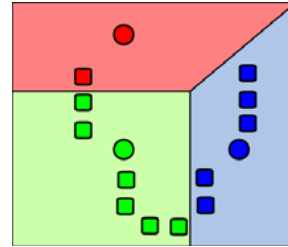
2. Assign each object to the cluster with the nearest centroid.



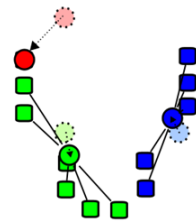
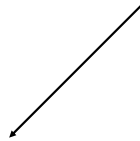
3. Compute each centroid as the mean of the objects assigned to it (go to 2)



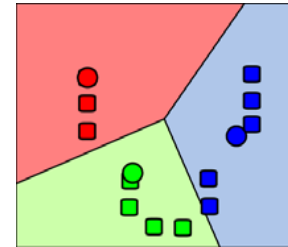
1. Select initial centroids at random



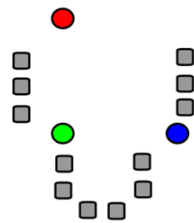
2. Assign each object to the cluster with the nearest centroid.



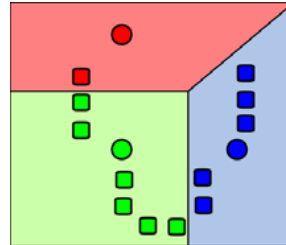
3. Compute each centroid as the mean of the objects assigned to it (go to 2)



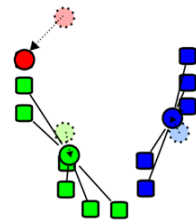
2. Assign each object to the cluster with the nearest centroid.



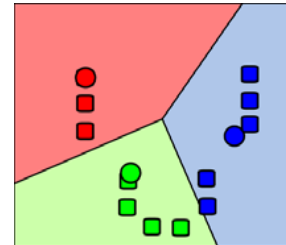
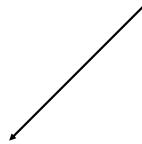
1. Select initial centroids at random



2. Assign each object to the cluster with the nearest centroid.



3. Compute each centroid as the mean of the objects assigned to it (go to 2)



2. Assign each object to the cluster with the nearest centroid.

Repeat previous 2 steps until no change

K-means Clustering

Given k :

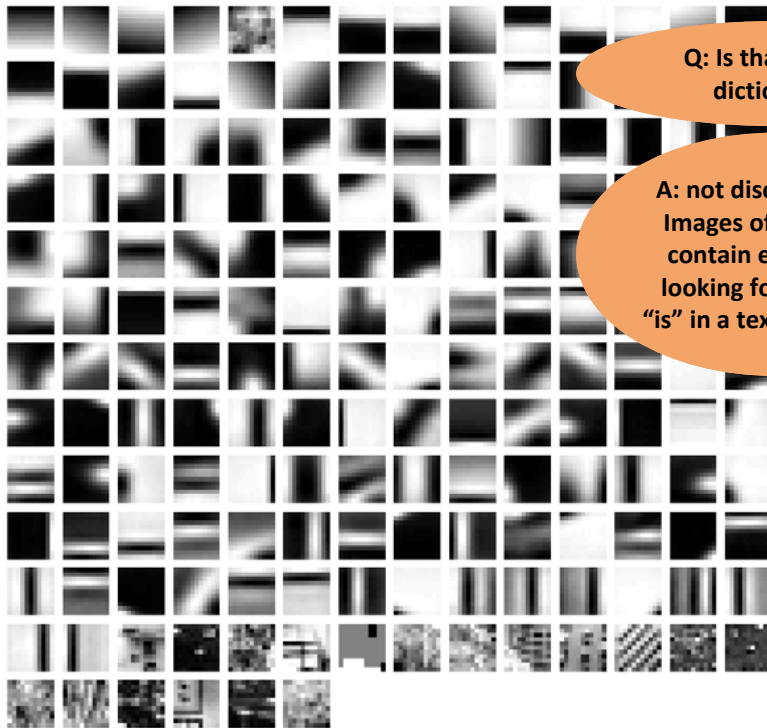
1. Select initial centroids at random.
2. Assign each object to the cluster with the nearest centroid.
3. Compute each centroid as the mean of the objects assigned to it.
4. Repeat previous 2 steps until no change.



*From what **data** should I learn the dictionary?*

- Dictionary should be learned on separate training set
- Provided the training set is sufficiently representative, the dictionary will be “universal”

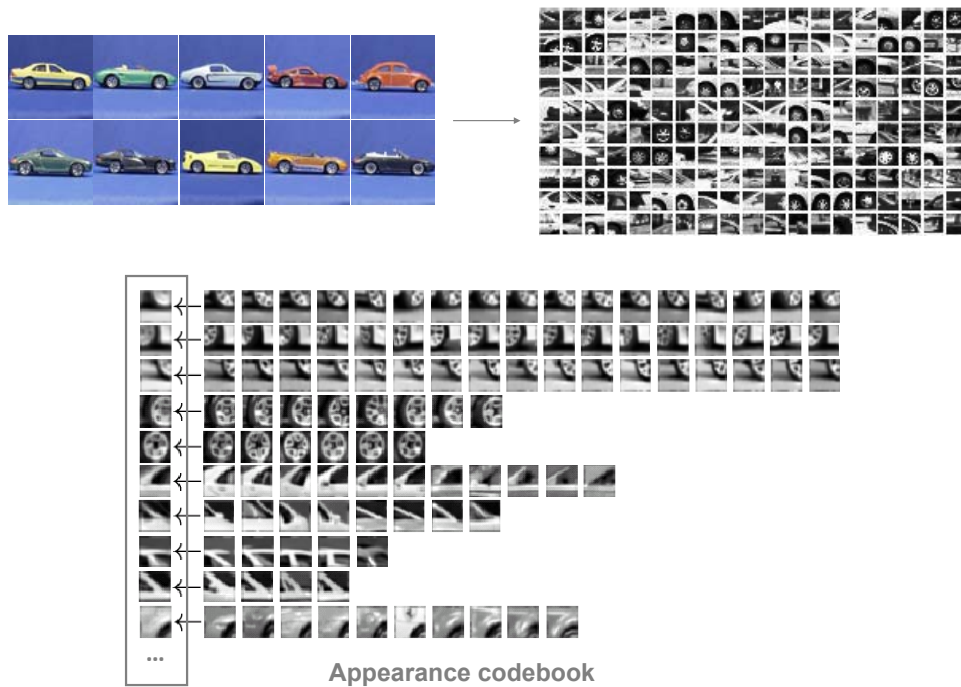
Example visual dictionary



Q: Is that a good dictionary?

A: not discriminative. Images of all classes contain edges. (like looking for the word "is" in a text document)

Example of a more discriminative dictionary



Source: B. Leibe

Here is an example of clusters in a more discriminative dictionary.

Now the clusters encode some more meaningful structure, such as different parts of a car.

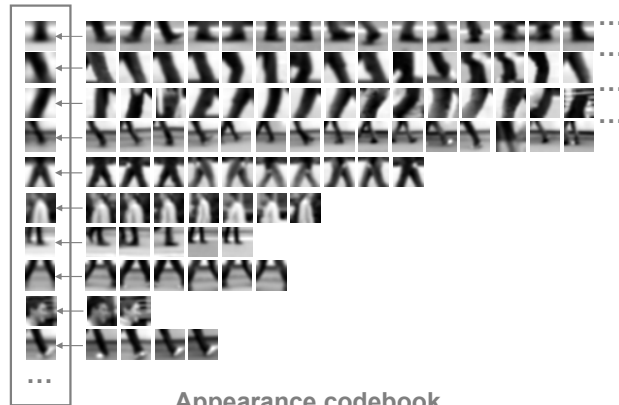
Still you can see the dictionary is not perfect for example we have multiple clusters with similar content. Note two different clusters of wheels, in one cluster the wheel is centered and in the other there is some offset.

We will have to work with this dictionary despite its problems.

For each cluster we show one patch representing its center. This is not the mean of all patches in the class, such a mean will be very blurry. Remember that the patches were matched based on their SIFT descriptor, not based on their intensities. The intensities can still vary a lot and if we just average intensities we will get a blurred image.

Instead the center image is chosen as one patch in the cluster whose SIFT descriptor is close to the mean SIFT descriptor.

Another dictionary



Appearance codebook

**Different dictionaries from
different training DB**

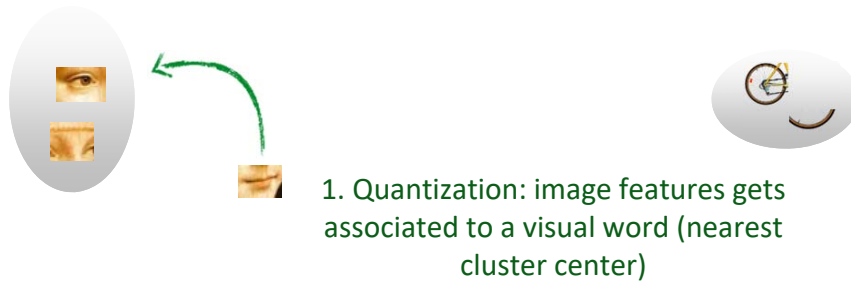
Source: B. Leibe

Dictionary Learning:
Learn Visual Words using clustering

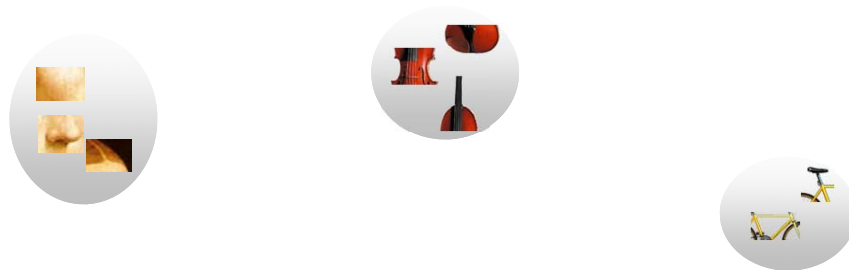
Encode:
build Bags-of-Words (BOW) vectors
for each image

Classify:
Train and test data using BOWs

Once we have defined the dictionary we want to take a new image and encode it, that is build for it a BOW representation.



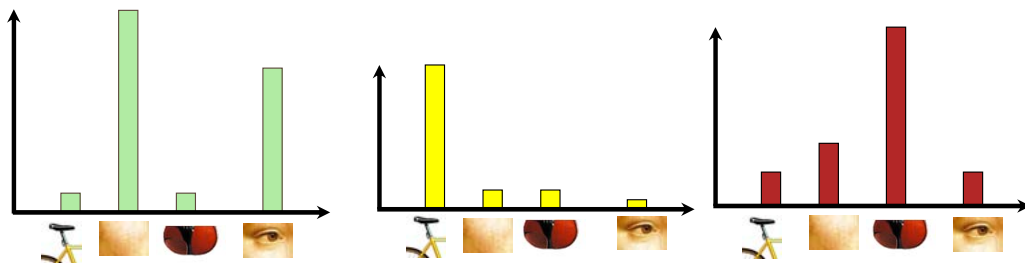
Encode:
build Bags-of-Words (BOW) vectors
for each image



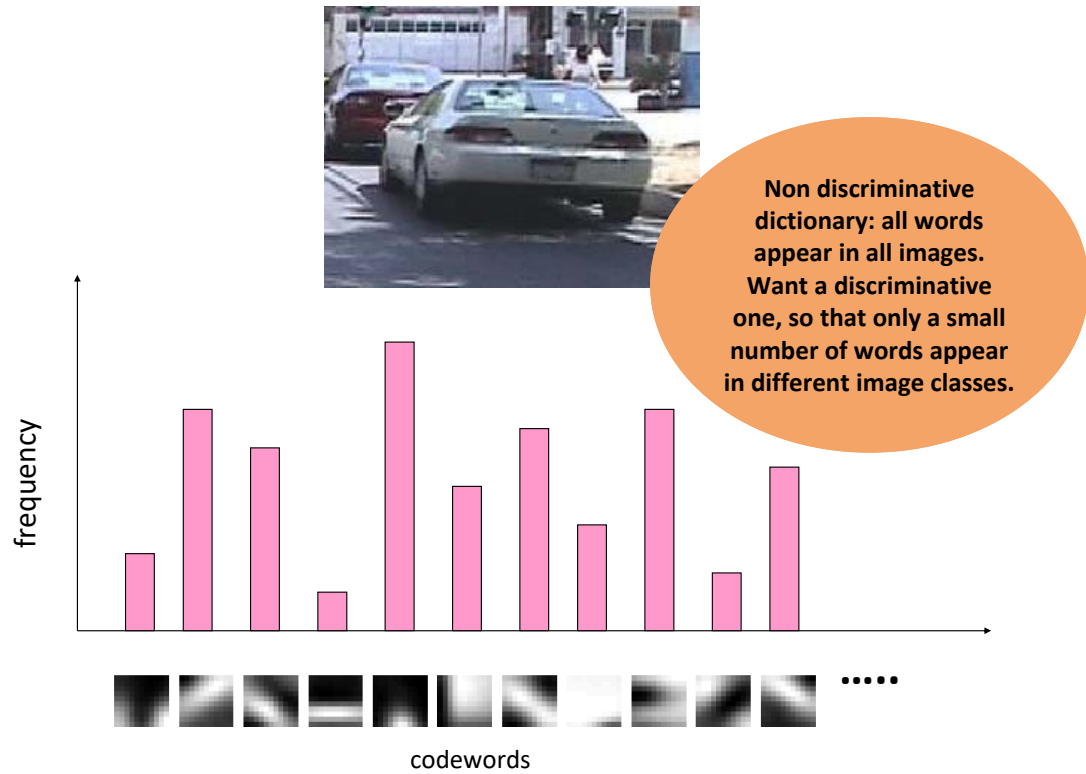
The first stage is to take the set of features we have extracted and check what word in the dictionary they represent. That is, check to what cluster they belong.

Encode:
build Bags-of-Words (BOW) vectors
for each image

2. Histogram: count the
number of visual word
occurrences



Then we build the BOW histogram representing the image: that is for each word, count how many times we detected this feature in the image.



If the dictionary is not discriminative all words will appear in all images. This will not help us in classification.

In a discriminative dictionary, only a small number of words appear in each image, and most other words do not appear (occurrence close to 0)

Dictionary Learning:

Learn Visual Words using clustering

Encode:

build Bags-of-Words (BOW) vectors
for each image

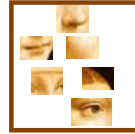
Classify:

Train and test data using BOWs

Dictionary Learning:

Learn Visual Words using clustering

1. extract features (e.g., SIFT) from images
2. Learn visual dictionary (e.g., K-means clustering)



Encode:

build Bags-of-Words (BOW) vectors
for each image

1. Quantization: image features gets associated to a visual word (nearest cluster center)
2. Histogram: count the number of visual word occurrences



Classify:

Train and test data using BOWs

1. Extract features from test image
2. Quantization: associated features to a visual word
3. Build histogram of test image
4. Classify, e.g using K nearest neighbors- find closest histogram in training set and copy its label

Dictionary Learning:

Learn Visual Words using clustering

1. extract features (e.g., SIFT) from images
2. Learn visual dictionary (e.g., K-means clustering)

Encode:

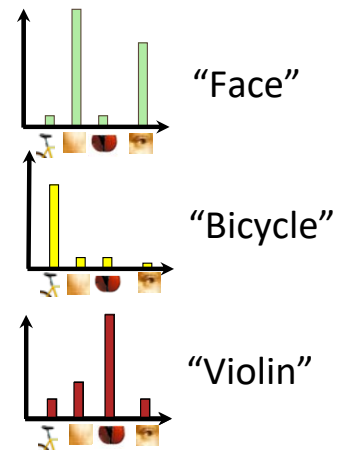
build Bags-of-Words (BOW) vectors
for each image

1. Quantization: image features gets associated to a visual word (nearest cluster center)
2. Histogram: count the number of visual word occurrences

Classify:

Train and test data using BOWs

1. Extract features from test image
2. Quantization: associated features to a visual word
3. Build histogram of test image
4. Classify, e.g. using K nearest neighbors- find closest histogram in training set and copy its label

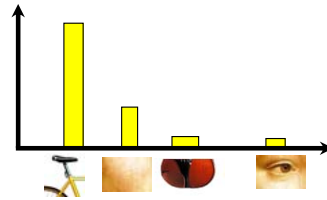




New test image



Extract feature histogram

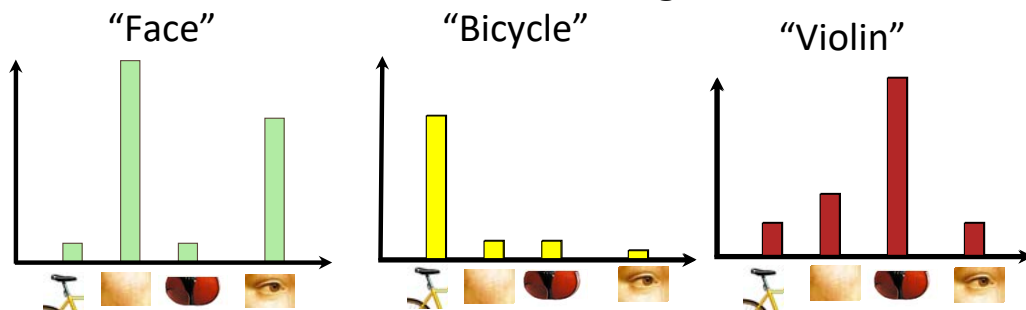


Note: Dictionary contains many “words”, In a histogram of a certain image only a small subset has non zero occurrence, this distribution helps us classify it.



Quarry test data:
What is the closest match?

Test Data Encoding



Dictionary contains a lot of word, collected over the full training DB.

The histogram of a given entry will probably have non zero entries only for a few words. Use this distribution for classification.

Dictionary Learning:

Learn Visual Words using clustering

1. extract features (e.g., SIFT) from images
2. Learn visual dictionary (e.g., **K-means** clustering)

Encode:

build Bags-of-Words (BOW) vectors
for each image

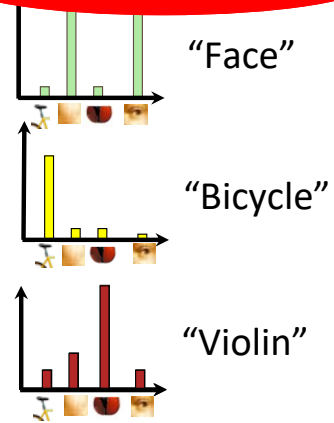
1. Quantization: image features gets associated to a visual word (nearest cluster center)
2. Histogram: count the number of visual word occurrences

Classify:

Train and test data using BOWs

1. Extract features from test image
2. Quantization: associated features to a visual word
3. Build histogram of test image
4. Classify, e.g. using **K nearest neighbors**- find closest histogram in training set and copy its label

Do not confuse



Handwritten notes in blue ink: 'K2D', 'Lip', and 'are with G'.